

Claude Code Quick Reference

AIA Copilot - Claude Code Training (August 2026 edition)

Claude Code is an agentic coding tool that runs in your terminal. It reads your codebase, plans changes, writes code, runs commands, and fixes its own errors. You provide intent; Claude provides implementation.

Installation (Native - no Node/npm)

The npm install path was deprecated January 2026. Use the native installer:

OS	Command
macOS / Linux	Run the claude.ai install script (<code>curl -fsSL https://claude.ai/install.sh</code> piped to <code>bash</code>) or <code>brew install claude-code</code>
Windows	<code>winget install Anthropic.ClaudeCode</code> (native - no WSL)
Linux pkg	apt / dnf / apk packages available

Verify: `claude --version -> v2.1.219+` for all course features. Confirm the approved identity, `/usage`, and `/model`; organization policy and provider support determine live access.

Models (August 2026)

Alias	Role
sonnet / Sonnet 5	Daily implementation and debugging; common subscription default
opus / Opus 5	Complex reasoning and architecture; Claude Code v2.1.219+
fable / Fable 5	Hardest, longest autonomous work when available
haiku / latest Haiku	Fast, low-cost mechanical work

Use `/model` to verify account availability and switch deliberately. Administrators and providers can restrict or remap available models.

Essential Commands

Command	What it does
<code>claude</code>	Start in current directory
<code>claude -p "..."</code>	Headless one-shot (CI) - pair with <code>--max-turns</code> , <code>--allowedTools</code>
<code>/clear</code>	Reset conversation; reloads skills, hooks, and extension configuration
<code>/compact</code>	Compress history, keep working
<code>/context</code>	Show what's in the window and what it costs
<code>/cost</code>	Token usage and dollars this session
<code>/model</code>	Switch models
<code>/doctor</code>	Full diagnostic; press <code>f</code> to auto-fix; flags unused skills, unused MCP servers, slow hooks
<code>/btw <q></code>	Ephemeral side question - no context pollution
<code>/memory</code>	View CLAUDE.md + Claude's auto-memory notes

Command	What it does
/permissions	Interactive permission rules menu
/skills /agents /plugin	Inventory skills / subagents / plugins
/keybindings	Customize keys -> ~/.claude/keybindings.json
/init	Bootstrap a CLAUDE.md from a codebase scan
/rewind	Open the rewind menu (same as double-Esc)

Keyboard Essentials

Keys	Action
Shift+Tab	Cycle permission modes: Default -> AcceptEdits -> Plan -> Auto
Esc Esc (double-tap)	Rewind menu - restore conversation, code, or both from a checkpoint
Esc (single)	Interrupt the current action
!command	Run shell command; Claude responds to the output directly (no second prompt)
@file	Load a specific file/folder/URL into context

The Four Permission Modes

Mode	Behavior
Default	Asks before edits and risky commands
AcceptEdits	Auto-approves file edits
Plan	Read-only - plans, never changes
Auto	Autonomous within your permission rules; ships with two-layer safety (injection probe + transcript classifier), rm -rf guards

Permission Rules (settings.json)

Three arrays: **deny**, **ask**, **allow** - **deny is evaluated first**, then ask, then allow. A deny match wins even when an allow rule also matches. Configure via `/permissions` (menu writes the JSON for you). Enterprise: `managed-settings.json` is locked; `managed-settings.d/*.json` merge alphabetically.

Checkpoints & Rewind

Claude Code checkpoints continuously. Double-Esc -> pick a checkpoint -> restore:

- **conversation** - bad instructions / derailed session
- **code** - bad file changes
- **both** - full undo

Rewind beats arguing with a contaminated context. For damage that predates your checkpoints (teammate's bad merge): describe what's wrong + point at `@reference` files + state what done looks like.

CLAUDE.md Scopes

Scope	File	Shared?
Personal (global)	<code>~/.claude/CLAUDE.md</code>	No
Personal (project)	<code>CLAUDE.local.md</code>	No (gitignore it)
Team	<code>CLAUDE.md</code> in repo	Yes
Subdirectory	<code>subfolder/CLAUDE.md</code>	Loaded in that folder

Keep under ~200 lines. **Auto memory:** Claude also keeps its own notes (build commands, insights) loaded alongside CLAUDE.md - you curate the rules, Claude curates its learnings.

CLI for MCP & Plugins

Command	What it does
<code>claude mcp add --transport http/stdio <name> <target></code>	Register an MCP server
<code>claude mcp login <name> / claude mcp logout <name></code>	Shell auth for MCP servers (week 28)
<code>claude mcp list / remove</code>	Inventory / prune
<code>claude plugin init <name> --with skills hooks</code>	Create a skills-directory plugin under <code>~/.claude/skills/<name>/</code>
<code>/plugin marketplace add <path-or-repo></code>	Add a marketplace
<code>/plugin install <name>@<marketplace></code>	Install a plugin; follow the summary if <code>/reload-plugins</code> is required
<code>claude plugin list / details</code>	Inspect installed state and contributed components
<code>/plugin disable / uninstall</code>	Deactivate or remove at an intentional scope

Context Discipline

- 1M-token window on Sonnet 5 - but treat it as a budget; compact around 50%.
- @ mentions load exactly what you need; vague scoping can cost 50K tokens.
- MCP ToolSearch defers schemas by default. Use `/context` to inspect actual loading, review `alwaysLoad` exceptions, and disable unused servers.
- Same task = keep context. New task = `/clear`.

Troubleshooting

Problem	Fix
Claude stops mid-task	<code>exit -> claude -> "Continue where you left off"</code>
Off the rails	Double-Esc, rewind to the last good checkpoint
Convention not followed	CLAUDE.md in root? <code>/clear</code> to reload
Skill won't fire	Pushier description; check <code>/skills</code> ; <code>/clear</code>
Hook won't fire	Exact-case matcher (<code>Write</code> not <code>write</code>); <code>/clear</code> ; <code>/doctor</code>

Problem	Fix
Slow/pricy	<code>/model</code> down a tier; <code>/compact</code> ; check <code>/cost</code>
Anything weird	<code>/doctor</code> , press <code>f</code>

Access Paths (August 2026)

Access path	Verify live
Pro or Max subscription	Correct identity, <code>/usage</code> , and <code>/model</code>
Team or Enterprise seat	Organization policy, seat allowance, approved models
Anthropic Console / API	Credential path and token-billing owner
Cloud provider	Provider authentication, model support, and budgets

Classroom rule: prove access in the live CLI before class; do not infer it from a remembered price table.

Resources

- Docs: <https://code.claude.com/docs>
- Changelog: <https://code.claude.com/docs/en/whats-new>
- GitHub Action: [anthropics/claude-code-action@v1](#)

(c) 2026 AIA Copilot - Claude Code Training